Name : Vaishnavi Eknath Avhad
Class : D20C
Roll No. : 2

Practical 8

Aim : Configuring a Cryptocurrency Wallet using Metamask and integrating it with a DApp.

Theory

MetaMask as a Web3 Provider
MetaMask acts as a bridge between web browsers and Ethereum-based blockchain networks. It injects a global Ethereum API object (window.ethereum) into the browser context, allowing decentralized web applications to request user wallet authorization, retrieve public addresses, and prompt users to sign banking transactions using their private keys.

Banking DApp Architecture & Front-End Integration
To handle banking workflows (depositing, withdrawing, and transferring Ether safely) from a user interface, client-side applications utilize libraries like Ethers.js or Web3.js:

- Provider: An abstraction for reading blockchain state data (e.g., querying account balances or checking contract treasury funds without gas fees).
- Signer: Represents an authorized user account capable of signing value-bearing transactions (e.g., depositing Ether or transferring funds).
- Contract Instance: A JavaScript representation of the smart contract initialized via its Deployed Address and ABI (Application Binary Interface).

Reentrancy Protection & Gas Optimization in Banking Smart Contracts
To ensure banking operations are executed in a safe and optimal manner:

- Checks-Effects-Interactions Pattern: State variables are updated in storage before external Ether transfers occur to prevent reentrancy attacks.
- Gas-Efficient State Updates: Reusable function execution logic avoids unnecessary computations and transaction failures on the EVM.

Program and Output

Smart Contract Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title OptimalBankingDApp
 * @dev Secure and gas-optimized
 * decentralized banking contract.
 */
contract OptimalBankingDApp {
    mapping(address => uint256) private
balances;

    event Deposit(address indexed account,
uint256 amount);
    event Withdrawal(address indexed account,
uint256 amount);
```

```
    event Transfer(
        address indexed sender,
        address indexed receiver,
        uint256 amount
    );

    // Deposit Ether into the banking contract
    function deposit() external payable {
        require(msg.value > 0, "Deposit amount
must be greater than zero");

        balances[msg.sender] += msg.value;

        emit Deposit(msg.sender, msg.value);
    }

    // Withdraw Ether safely using
Checks-Effects-Interactions pattern
    function withdraw(uint256 _amount) external
{
        require(_amount > 0, "Withdrawal amount
must be greater than zero");
        require(
            balances[msg.sender] >= _amount,
            "Insufficient bank balance"
        );

        // Effect
        balances[msg.sender] -= _amount;

        // Interaction
        (bool success, ) =
payable(msg.sender).call{value: _amount}("");
        require(success, "Ether transfer failed");

        emit Withdrawal(msg.sender, _amount);
    }

    // Optimal peer-to-peer internal transfer
between bank accounts
    function transferFunds(address payable _to,
uint256 _amount) external {
        require(_to != address(0), "Invalid recipient
address");
        require(_to != msg.sender, "Cannot transfer
to self");
        require(_amount > 0, "Transfer amount
must be greater than zero");
        require(balances[msg.sender] >= _amount,
"Insufficient balance");

        balances[msg.sender] -= _amount;
        balances[_to] += _amount;

        emit Transfer(msg.sender, _to, _amount);
    }

    // Read user balance
    function getBalance(address _account)
external view returns (uint256) {
        return balances[_account];
    }
}
```

DApp Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
content="width=device-width, initial-scale=1.0">
    <title>Decentralized Banking DApp</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva,
Verdana, sans-serif;
            margin: 30px;
            background-color: #f0f2f5;
        }
        .card {
            background: white;
            padding: 25px;
            border-radius: 10px;
            max-width: 500px;
```

```
        box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
        margin: auto;
      }
      button {
        background-color: #107c41;
        color: white;
        border: none;
        padding: 10px 16px;
        font-size: 14px;
        border-radius: 6px;
        cursor: pointer;
        font-weight: 600;
        margin-top: 5px;
      }
      button:hover {
        background-color: #0b5a2f;
      }
      .btn-connect {
        background-color: #f6851b;
      }
      .btn-connect:hover {
        background-color: #e2761b;
      }
      input {
        padding: 8px 12px;
        width: calc(100% - 28px);
        margin: 6px 0 12px 0;
        border: 1px solid #ccc;
        border-radius: 6px;
      }
      .status {
        margin-top: 10px;
        font-size: 14px;
        color: #333;
        word-break: break-all;
      }
      .balance-box {
        background: #e8f5e9;
        padding: 12px;
        border-radius: 6px;
        font-weight: bold;
        font-size: 16px;
        color: #1b5e20;
        margin: 15px 0;
      }
    </style>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/ethers/5.7.
2/ethers.umd.min.js"></script>
</head>
<body>
  <div class="card">
    <h2>Decentralized Bank Portal</h2>
    <button class="btn-connect" id="connectBtn"
onclick="connectWallet()">
      Connect MetaMask Wallet
    </button>
    <div class="status" id="accountStatus">
      Wallet Status: Not Connected
    </div>
    <div class="balance-box"
id="bankBalanceDisplay">
      Vault Balance: 0.0 ETH
    </div>
    <hr style="margin: 20px 0;">
    <h3>Banking Operations</h3>
    <label><b>Deposit ETH:</b></label>
    <input
      type="number"
      step="0.01"
      id="depositInput"
      placeholder="Amount in ETH"
    >
    <button onclick="depositETH()">Deposit to
Bank</button>
    <br><br>
    <label><b>Withdraw ETH:</b></label>
    <input
      type="number"
      step="0.01"
      id="withdrawInput"
      placeholder="Amount in ETH"
    >
    <button onclick="withdrawETH()">Withdraw
to Wallet</button>
    <br><br>
    <label><b>Transfer Funds
(Internal):</b></label>
    <input
      type="text"
      id="recipientInput"
      placeholder="Recipient Address (0x...)"
    >
    <input
      type="number"
      step="0.01"
      id="transferInput"
      placeholder="Amount in ETH"
```

```
      >
      <button onclick="transferETH()">Transfer
Funds</button>
      <div class="status" id="txStatus"
style="margin-top: 15px; font-weight: 600;"></div>
   </div>
   <script>
      const contractAddress =
"0xDA0bab807633f07f013f94DD0E6A4F96F8742B
53";
      const contractABI = [
         "function deposit() external payable",
         "function withdraw(uint256 _amount)
external",
         "function transferFunds(address payable _to,
uint256 _amount) external",
         "function getBalance(address _account)
external view returns (uint256)"
      ];
      let provider, signer, contract, userAccount;
      async function connectWallet() {
         if (typeof window.ethereum !== 'undefined') {
            try {
               const accounts = await
window.ethereum.request({
                  method: 'eth_requestAccounts'
               });
               userAccount = accounts[0];

document.getElementById('accountStatus').innerText
=
                  "Account: " + userAccount;
               provider = new
ethers.providers.Web3Provider(window.ethereum);
               signer = provider.getSigner();
               contract = new ethers.Contract(
                  contractAddress,
                  contractABI,
                  signer
               );
               updateBalance();
            } catch (err) {
               console.error("Connection Error:", err);
            }
         } else {
            alert("MetaMask extension not detected!");
         }
      }
      async function updateBalance() {
         if (!contract || !userAccount) return;
         try {
            const rawBalance = await
contract.getBalance(userAccount);
            const formatted =
ethers.utils.formatEther(rawBalance);

document.getElementById('bankBalanceDisplay').inn
erText =
               `Vault Balance: ${formatted} ETH`;
         } catch (err) {
            console.error("Balance Fetch Error:", err);
         }
      }
      async function depositETH() {
         const val =
document.getElementById('depositInput').value;
         if (!val || val <= 0) {
            return alert("Enter a valid ETH amount");
         }
         try {

document.getElementById('txStatus').innerText =
               "Processing Deposit...";
            const tx = await contract.deposit({
               value: ethers.utils.parseEther(val)
            });
            await tx.wait();

document.getElementById('txStatus').innerText =
               "Deposit Successful! TX Hash: " +
tx.hash;
            updateBalance();
         } catch (err) {
            console.error(err);

document.getElementById('txStatus').innerText =
               "Deposit failed or cancelled.";
         }
      }
      async function withdrawETH() {
         const val =
document.getElementById('withdrawInput').value;
         if (!val || val <= 0) {
            return alert("Enter a valid ETH amount");
         }
         try {

document.getElementById('txStatus').innerText =
```

```
            "Processing Withdrawal...";
          const tx = await contract.withdraw(
            ethers.utils.parseEther(val)
          );
          await tx.wait();

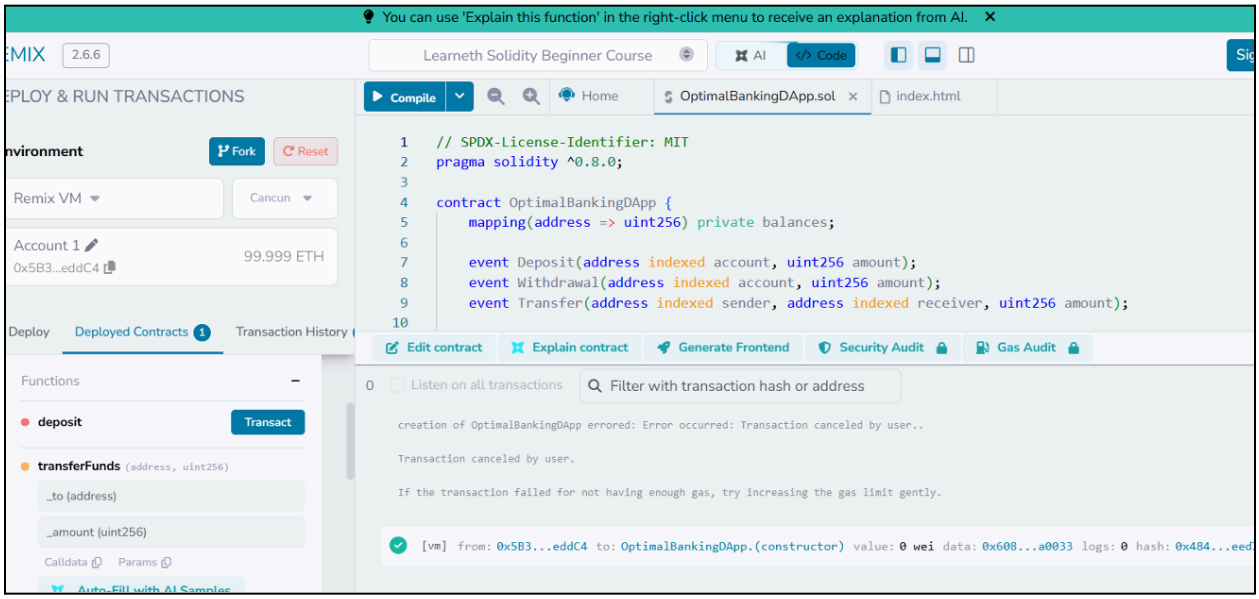
document.getElementById('txStatus').innerText =
            "Withdrawal Successful! TX Hash: " +
tx.hash;
          updateBalance();
        } catch (err) {
          console.error(err);

document.getElementById('txStatus').innerText =
            "Withdrawal failed.";
        }
      }
      async function transferETH() {
        const recipient =
document.getElementById('recipientInput').value;
        const val =
document.getElementById('transferInput').value;
        if (!recipient || !val || val <= 0) {
          return alert("Enter valid recipient and
amount");
        }
        try {

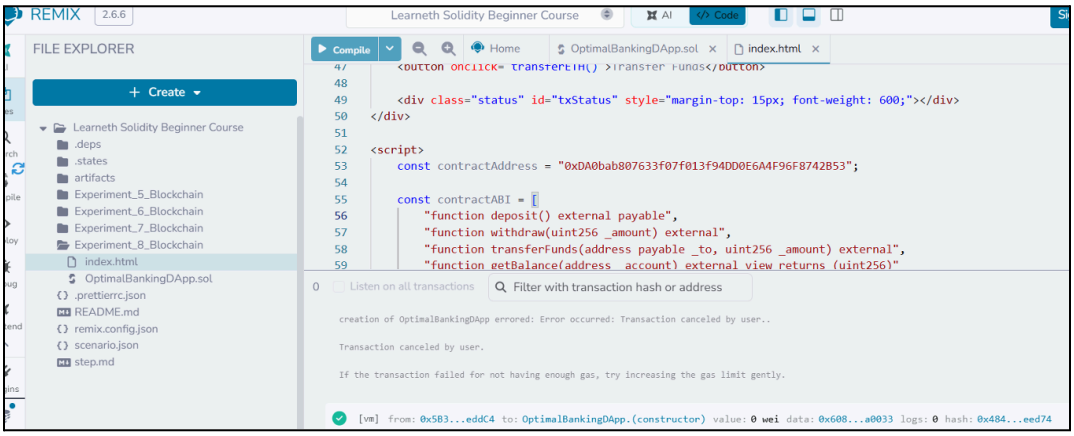
document.getElementById('txStatus').innerText =
            "Processing Internal Transfer...";
          const tx = await contract.transferFunds(
            recipient,
            ethers.utils.parseEther(val)
          );
          await tx.wait();

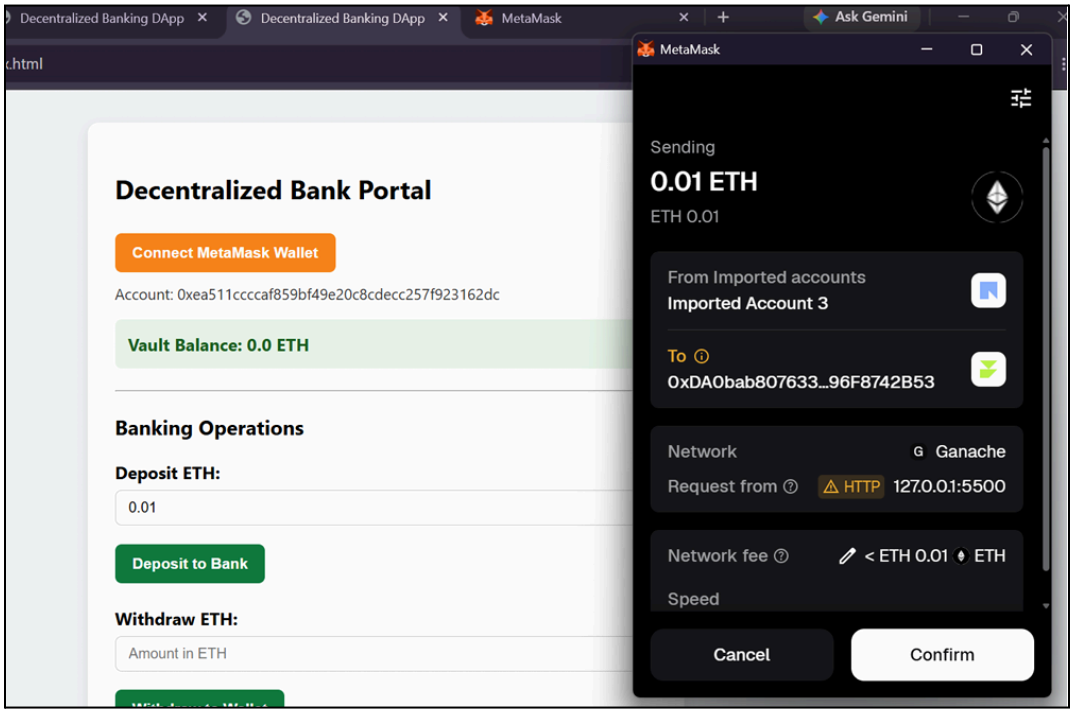
document.getElementById('txStatus').innerText =
            "Transfer Successful! TX Hash: " +
tx.hash;
          updateBalance();
        } catch (err) {
          console.error(err);

document.getElementById('txStatus').innerText =
            "Transfer failed.";
        }
      }
    </script>
</body>
</html>
```









Conclusion

A decentralized Banking Application (DApp) was successfully implemented, deployed, and integrated with MetaMask wallet. By using Ethers.js and injected Web3 provider instances (window.ethereum), seamless interaction was established between client-side JavaScript and smart contracts on the Ethereum network. Practical operations including wallet account linking, state reading without gas fees, and signing transactions for contract balance deposits were verified.